Documentação Técnica
Redefinição de Senha (Forgot Password)

Projeto:	Barber System
Tecnologia:	Spring Boot + Spring Security + JWT + MySQL
Funcionalidade:	Recuperação de senha via e-mail (Gmail)
Data:	Setembro/2026

1. Visão Geral

Esta documentação descreve, de forma completa e detalhada, como implementar a funcionalidade de **"Esqueci a senha"** no sistema Barber System.

O fluxo adotado é o padrão de mercado mais seguro:

1. O usuário informa o e-mail.
2. O sistema gera um **token único** com validade de 30 minutos.
3. É enviado um e-mail contendo um **link de redefinição**.
4. O usuário clica no link, define a nova senha e o token é invalidado.

■ **Importante:** Nunca envie a nova senha diretamente por e-mail. Sempre utilize token + link.

2. Análise do Projeto Atual

Antes de implementar, é fundamental entender as particularidades do Barber System:

Característica	Situação no Projeto	Impacto na implementação
Tipos de usuário	3 entidades separadas (Cliente, Barbeiro, Administrador)	O token deve funcionar para qualquer tipo
Login	Endpoint único /login que busca nas 3 tabelas	O forgot-password também deve buscar nas 3 tabelas
Criptografia de senha	Já utiliza BCryptPasswordEncoder	Reutilizar o mesmo PasswordEncoder
Segurança	JWT + Spring Security (stateless)	Liberar 2 novos endpoints públicos
Dependência de e-mail	Não existe no pom.xml	Adicionar spring-boot-starter-mail
Alteração de senha existente	Existe alterarSenha (exige senha atual + usuário logado)	Nova funcionalidade é para usuário não logado

3. Passo a Passo de Implementação

3.1. Adicionar a dependência de e-mail

No arquivo `pom.xml`, dentro da tag <dependencies>, adicione:

```
<!-- Envio de e-mail -->
<dependency>
    <groupId>org.springframework.boot</groupId>
```

```
    <artifactId>spring-boot-starter-mail</artifactId>
</dependency>
```

Em seguida, atualize as dependências:

```
./mvnw clean install
```

3.2. Configurar o Gmail no application.properties

Abra o arquivo `src/main/resources/application.properties` e adicione no final:

```
# ==========================================
# Configuração de e-mail (Gmail)
# ==========================================
spring.mail.host=smtp.gmail.com
spring.mail.port=587
spring.mail.username=seu-email@gmail.com
spring.mail.password=sua-senha-de-app
spring.mail.properties.mail.smtp.auth=true
spring.mail.properties.mail.smtp.starttls.enable=true
spring.mail.properties.mail.smtp.starttls.required=true

# URL base que aparecerá no link do e-mail
# Em produção, troque pelo domínio real do frontend
app.reset-password-url=http://localhost:3000/redefinir-senha
```

Como gerar a Senha de App do Gmail (obrigatório):

1. Acesse sua conta Google → **Segurança**.
2. Ative a **Verificação em 2 etapas** (caso ainda não esteja ativa).
3. Procure por **Senhas de app**.
4. Crie uma senha de app (selecione “E-mail” ou “Outro”).
5. Cole a senha gerada no campo `spring.mail.password`.

■ **Atenção:** Não utilize a senha normal da conta Google. O Gmail bloqueia autenticação com senha comum.

3.3. Criar a Entidade do Token de Reset

Como existem **três tipos de usuário**, o token armazena apenas o **e-mail**. Na hora de redefinir a senha, o sistema descobre em qual tabela o e-mail está.

Crie o arquivo: `src/main/java/com/zheiry/barbersystem/entities/PasswordResetTokenEntity.java`

```
package com.zheiry.barbersystem.entities;

import jakarta.persistence.*;
import java.time.LocalDateTime;

@Entity
@Table(name = "password_reset_token")
public class PasswordResetTokenEntity {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true)
    private String token;

    @Column(nullable = false)
    private String email;          // e-mail do usuário (qualquer tipo)

    @Column(nullable = false)
    private LocalDateTime expiryDate;

    public PasswordResetTokenEntity() {}

    // Getters e Setters
    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
```

```
    public String getToken() { return token; }
    public void setToken(String token) { this.token = token; }

    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }

    public LocalDateTime getExpiryDate() { return expiryDate; }
    public void setExpiryDate(LocalDateTime expiryDate) {
        this.expiryDate = expiryDate;
    }
}
```

Com `spring.jpa.hibernate.ddl-auto=update` a tabela será criada automaticamente.

3.4. Criar o Repository do Token

Crie o arquivo: `src/main/java/com/zheiry/barbersystem/repositories/PasswordResetTokenRepository.java`

```
package com.zheiry.barbersystem.repositories;

import com.zheiry.barbersystem.entities.PasswordResetTokenEntity;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
import java.util.Optional;

@Repository
public interface PasswordResetTokenRepository
        extends JpaRepository<PasswordResetTokenEntity, Long> {

    Optional<PasswordResetTokenEntity> findByToken(String token);

    void deleteByEmail(String email);   // limpa tokens antigos
}
```

3.5. Criar o Serviço de Envio de E-mail

Crie o arquivo: `src/main/java/com/zheiry/barbersystem/services/EmailService.java`

```
package com.zheiry.barbersystem.services;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.mail.SimpleMailMessage;
import org.springframework.mail.javamail.JavaMailSender;
import org.springframework.stereotype.Service;

@Service
public class EmailService {

    private final JavaMailSender mailSender;

    @Value("${spring.mail.username}")
    private String from;

    public EmailService(JavaMailSender mailSender) {
        this.mailSender = mailSender;
    }

    public void enviarEmailRedefinicaoSenha(String para, String link) {
        SimpleMailMessage message = new SimpleMailMessage();
        message.setFrom(from);
        message.setTo(para);
        message.setSubject("Redefinição de Senha - Barber System");
        message.setText(
            "Olá,\n\n" +
            "Recebemos uma solicitação para redefinir a senha "
            + "da sua conta.\n\n" +
            "Clique no link abaixo para criar uma nova senha:\n" +
            link + "\n\n" +
            "Este link é válido por 30 minutos.\n\n" +
            "Se você não solicitou essa alteração, ignore este "
```

```
            + "e-mail.\n\n" +
            "Atenciosamente,\nEquipe Barber System"
        );
        mailSender.send(message);
    }
}
```

Dica: Para e-mails mais elaborados (HTML), utilize `MimeMessageHelper` no lugar de `SimpleMailMessage`.

3.6. Criar o Serviço Principal de Reset de Senha

Este é o núcleo da funcionalidade. Ele precisa localizar o e-mail nas **três tabelas**.

Crie o arquivo: `src/main/java/com/zheiry/barbersystem/services/PasswordResetService.java`

```
package com.zheiry.barbersystem.services;

import com.zheiry.barbersystem.entities.*;
import com.zheiry.barbersystem.exception.BadRequestException;
import com.zheiry.barbersystem.exception.ResourceNotFoundException;
import com.zheiry.barbersystem.repositories.*;
import jakarta.transaction.Transactional;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.stereotype.Service;
import java.time.LocalDateTime;
import java.util.Optional;
import java.util.UUID;

@Service
public class PasswordResetService {

    private final ClienteRepository clienteRepository;
    private final BarbeiroRepository barbeiroRepository;
    private final AdministradorRepository administradorRepository;
    private final PasswordResetTokenRepository tokenRepository;
    private final EmailService emailService;
    private final PasswordEncoder passwordEncoder;

    @Value("${app.reset-password-url}")
    private String resetPasswordUrl;

    public PasswordResetService(
            ClienteRepository clienteRepository,
            BarbeiroRepository barbeiroRepository,
            AdministradorRepository administradorRepository,
            PasswordResetTokenRepository tokenRepository,
            EmailService emailService,
            PasswordEncoder passwordEncoder) {
        this.clienteRepository = clienteRepository;
        this.barbeiroRepository = barbeiroRepository;
        this.administradorRepository = administradorRepository;
        this.tokenRepository = tokenRepository;
        this.emailService = emailService;
        this.passwordEncoder = passwordEncoder;
    }

    /** 1º passo: Usuário informa o e-mail */
    @Transactional
    public void solicitarRedefinicao(String email) {
        boolean existe = clienteRepository.findByEmail(email).isPresent()
            || barbeiroRepository.findByEmail(email).isPresent()
            || administradorRepository.findByEmail(email).isPresent();

        if (!existe) {
            // Por segurança, não revelamos se o e-mail existe
            return;
        }

        tokenRepository.deleteByEmail(email);

        String token = UUID.randomUUID().toString();
```

```
        PasswordResetTokenEntity resetToken =
            new PasswordResetTokenEntity();
        resetToken.setToken(token);
        resetToken.setEmail(email);
        resetToken.setExpiryDate(
            LocalDateTime.now().plusMinutes(30));

        tokenRepository.save(resetToken);

        String link = resetPasswordUrl + "?token=" + token;
        emailService.enviarEmailRedefinicaoSenha(email, link);
    }

    /** 2º passo: Redefinir a senha com o token */
    @Transactional
    public void redefinirSenha(String token, String novaSenha,
                               String confirmaNovaSenha) {

        if (!novaSenha.equals(confirmaNovaSenha)) {
            throw new BadRequestException(
                "Nova senha e confirmação não conferem!");
        }

        PasswordResetTokenEntity resetToken =
            tokenRepository.findByToken(token)
                .orElseThrow(() -> new BadRequestException(
                    "Token inválido ou já utilizado"));

        if (resetToken.getExpiryDate()
                .isBefore(LocalDateTime.now())) {
            tokenRepository.delete(resetToken);
            throw new BadRequestException(
                "Token expirado. Solicite uma nova redefinição.");
        }

        String email = resetToken.getEmail();
        String senhaCriptografada =
            passwordEncoder.encode(novaSenha);

        Optional<ClienteEntity> cliente =
            clienteRepository.findByEmail(email);
        if (cliente.isPresent()) {
            cliente.get().setSenha(senhaCriptografada);
            clienteRepository.save(cliente.get());
            tokenRepository.delete(resetToken);
            return;
        }

        Optional<BarbeiroEntity> barbeiro =
            barbeiroRepository.findByEmail(email);
        if (barbeiro.isPresent()) {
            barbeiro.get().setSenha(senhaCriptografada);
            barbeiroRepository.save(barbeiro.get());
            tokenRepository.delete(resetToken);
            return;
        }

        Optional<AdministradorEntity> admin =
            administradorRepository.findByEmail(email);
        if (admin.isPresent()) {
            admin.get().setSenha(senhaCriptografada);
            administradorRepository.save(admin.get());
            tokenRepository.delete(resetToken);
            return;
        }

        throw new ResourceNotFoundException(
            "Usuário não encontrado para este token");
    }
}
```

Pré-requisito nos Repositórios: Certifique-se de que os repositórios de Barbeiro e Administrador possuem o método:

```
Optional<BarbeiroEntity> findByEmail(String email);
Optional<AdministradorEntity> findByEmail(String email);
```

3.7. Criar os DTOs

ForgotPasswordDto — `src/main/java/com/zheiry/barbersystem/dto/ForgotPasswordDto.java`

```
package com.zheiry.barbersystem.dto;

import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.NotBlank;

public record ForgotPasswordDto(
    @NotBlank(message = "E-mail é obrigatório")
    @Email(message = "E-mail inválido")
    String email
) {}
```

ResetPasswordDto — `src/main/java/com/zheiry/barbersystem/dto/ResetPasswordDto.java`

```
package com.zheiry.barbersystem.dto;

import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Pattern;

public record ResetPasswordDto(

    @NotBlank(message = "Token é obrigatório")
    String token,

    @NotBlank(message = "Nova senha é obrigatória")
    @Pattern(
        regexp = "^(?=.*[a-z])(?=.*[A-Z])(?=.*\\d)"
            + "(?=.*[@$!%*?&.#_-])"
            + "[A-Za-z\\d@$!%*?&.#_-]{8,}$",
        message = "A nova senha deve conter no mínimo 8 "
            + "caracteres, 1 letra maiúscula, 1 letra "
            + "minúscula, 1 número e 1 caractere especial"
    )
    String novaSenha,

    @NotBlank(message = "Confirmação da senha é obrigatória")
    String confirmaNovaSenha
) {}
```

3.8. Criar o Controller

Crie o arquivo: `src/main/java/com/zheiry/barbersystem/controller/PasswordResetController.java`

```
package com.zheiry.barbersystem.controller;

import com.zheiry.barbersystem.dto.ForgotPasswordDto;
import com.zheiry.barbersystem.dto.ResetPasswordDto;
import com.zheiry.barbersystem.services.PasswordResetService;
import jakarta.validation.Valid;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/auth")
public class PasswordResetController {

    private final PasswordResetService passwordResetService;

    public PasswordResetController(
            PasswordResetService passwordResetService) {
        this.passwordResetService = passwordResetService;
    }

    /** POST /auth/forgot-password */
    @PostMapping("/forgot-password")
    public ResponseEntity<String> forgotPassword(
```

```
            @RequestBody @Valid ForgotPasswordDto dto) {
        passwordResetService.solicitarRedefinicao(dto.email());
        return ResponseEntity.ok(
            "Se o e-mail existir em nossa base, você "
            + "receberá um link de redefinição.");
    }

    /** POST /auth/reset-password */
    @PostMapping("/reset-password")
    public ResponseEntity<String> resetPassword(
            @RequestBody @Valid ResetPasswordDto dto) {
        passwordResetService.redefinirSenha(
            dto.token(), dto.novaSenha(),
            dto.confirmaNovaSenha());
        return ResponseEntity.ok(
            "Senha redefinida com sucesso!");
    }
}
```

3.9. Liberar os endpoints na Security

Abra o arquivo `SecurityConfiguracao.java`. Dentro do bloco `.authorizeHttpRequests(...)`, adicione as linhas logo após o `/login`:

```
.requestMatchers("/login").permitAll()

// Novos endpoints públicos de redefinição de senha
.requestMatchers("/auth/forgot-password").permitAll()
.requestMatchers("/auth/reset-password").permitAll()

// Cadastro de cliente é público...
.requestMatchers(HttpMethod.POST, "/clientes").permitAll()
```

■ **Importante:** Sem essa liberação, o Spring Security retornará **401 Unauthorized** ou **403 Forbidden**.

4. Fluxo Completo da Funcionalidade

Passo 1 — Solicitar redefinição

Frontend envia `POST /auth/forgot-password` com o e-mail do usuário.

Backend gera token UUID, salva no banco (validade 30 min) e envia e-mail via Gmail.

Resposta sempre genérica (não revela se o e-mail existe).

Passo 2 — Usuário recebe o e-mail

O e-mail contém o link: `http://localhost:3000/redefinir-senha?token=abc-123`

Passo 3 — Redefinir a senha

Frontend captura o token da URL e envia `POST /auth/reset-password` com token + nova senha.

Backend valida o token, atualiza a senha (BCrypt) na tabela correta e invalida o token.

5. Endpoints

Método	Endpoint	Auth	Body	Descrição
POST	/auth/forgot-password	Pública	{ "email": "..." }	Solicita o envio do link
POST	/auth/reset-password	Pública	{ "token", "novaSenha", "confirmaNovaSenha" }	Redefine a senha

6. Boas Práticas de Segurança Aplicadas

Prática	Motivo
Token com expiração de 30 minutos	Reduz a janela de ataque
Token de uso único	Impede reutilização após a troca de senha
Mensagem genérica no forgot-password	Evita enumeração de usuários
Uso de BCryptPasswordEncoder	Senha nunca é salva em texto puro
Validação de força da senha (regex)	Garante senha forte
HTTPS em produção	Protege o token que trafega no link
Rate limiting (recomendado)	Evita spam de e-mails

7. Checklist de Implementação

- ■ 1. Adicionar spring-boot-starter-mail no pom.xml
- ■ 2. Configurar Gmail no application.properties
- ■ 3. Criar PasswordResetTokenEntity
- ■ 4. Criar PasswordResetTokenRepository
- ■ 5. Criar EmailService
- ■ 6. Criar PasswordResetService
- ■ 7. Criar ForgotPasswordDto e ResetPasswordDto
- ■ 8. Criar PasswordResetController
- ■ 9. Liberar /auth/forgot-password e /auth/reset-password na Security
- ■ 10. Garantir findByEmail em BarbeiroRepository e AdministradorRepository

8. Testando com Postman / Insomnia

8.1. Solicitar redefinição

```
POST http://localhost:8080/auth/forgot-password
Content-Type: application/json

{
  "email": "cliente@gmail.com"
}
```

Resposta esperada:

```
"Se o e-mail existir em nossa base, você receberá um link de redefinição."
```

8.2. Redefinir a senha

```
POST http://localhost:8080/auth/reset-password
Content-Type: application/json

{
  "token": "cole-aqui-o-token-recebido-no-email",
  "novaSenha": "NovaSenha@123",
  "confirmaNovaSenha": "NovaSenha@123"
}
```

Resposta esperada:

```
"Senha redefinida com sucesso!"
```

9. Possíveis Melhorias Futuras

- Enviar e-mail em formato **HTML** (mais profissional).
- Implementar **código de 6 dígitos** como alternativa ao link.

- Adicionar **rate limiting** no endpoint de forgot-password.
- Criar um job agendado para limpar tokens expirados periodicamente.
- Enviar notificação de segurança após a troca de senha.

10. Resumo dos Arquivos Criados/Alterados

Ação	Arquivo
Alterar	pom.xml
Alterar	application.properties
Alterar	SecurityConfiguracao.java
Criar	entities/PasswordResetTokenEntity.java
Criar	repositories/PasswordResetTokenRepository.java
Criar	services/EmailService.java
Criar	services/PasswordResetService.java
Criar	dto/ForgotPasswordDto.java
Criar	dto/ResetPasswordDto.java
Criar	controller/PasswordResetController.java

Documento gerado para o projeto **Barber System**
Versão 1.0 — Setembro/2026